

API CLI MCP

FRONTMATTER

title: API CLI MCP
video no: 15 of 30
series: Building an AI Business From Scratch
voice_reference: ../../Input/4. Resources/2.
scripting_resources/voice_reference/nate_herk_voice_reference.md
video_type: concept
hook_type: Curiosity-Gap
script_versions: A + B
status: DRAFT

REFERENCE VIDEOS

- MCP vs. CLI Explained in 5 Minutes (for beginners) (<https://www.youtube.com/watch?v=RSN82iM3jWs>) -- Independent creator -- Watch for the plain-English framing of what makes MCP different from CLI in an AI context
- CLI vs MCP: How AI Agents Choose the Right Tool for the Job (<https://www.youtube.com/watch?v=g9JIUM0MHgQ>) -- Independent creator -- Watch for the decision framework: which access method fits which use case
- CLAUDE CODE FULL COURSE 4 HOURS: Build & Sell (2026) (<https://www.youtube.com/watch?v=QoQBzR1NIql>) -- Nick Saraev -- Watch for how Nick moves between API and CLI depending on the task -- real-world usage patterns

THE ONE THING

API, CLI, and MCP are three ways to give AI access to external tools and data -- and picking the right one for the right job is what separates quick scripts from real systems.

HOOK

API. CLI. MCP. You have heard all three. Nobody explains what they actually are or when to use which one. This is that video.

TARGET VIEWER

Someone building with AI who keeps running into these terms and wants a clear, non-technical explanation of what each one is and when they would actually reach for it.

PAYOFF

After watching, the viewer can explain what API, CLI, and MCP are, give a real example of each, and make a confident decision about which to use for a given automation task.

BEAT STRUCTURE

Beat -- Why This Matters

Point: Once you move beyond using Claude through its chat interface, you need to know how to connect Claude to external tools and data. API, CLI, and MCP are your three options.

Visual: Diagram: Claude in the centre, three connection types branching out to external tools -- API, CLI, MCP each pointing to different tools/services

Target words: ~100w

Beat -- API -- The Standard Connection

Point: An API is an HTTP interface -- you send a request with text, you get back a response. Every AI model has one. It is how you connect Claude to your own apps and scripts.

Visual: Simple code snippet: one API call to Claude -- show the request structure and response structure. No framework, just curl or Python requests.

Target words: ~170w

Beat -- CLI -- Fast, Local, Terminal

Point: CLI tools are commands you run in your terminal. Claude Code itself is a CLI. CLIs are fast, reliable, and cheap to run -- and AI agents can call them just like a human would.

Visual: Terminal: running a CLI tool (like git or a custom script) from inside a Claude Code session -- Claude calls the tool, gets back output, uses it in its response

Target words: ~160w

Beat -- MCP -- Structured Tool Access for AI

Point: MCP (Model Context Protocol) is Anthropic's standard for connecting AI to external data and tools in a structured, consistent way. An MCP server exposes tools that Claude can call by name.

Visual: Diagram: MCP server with three tools listed -- Claude calling a tool by name, getting structured output back. Then a quick terminal demo of an MCP-connected Claude session.

Target words: ~180w

Beat -- When to Use Which

Point: API for building apps that call Claude. CLI for local automation and simple agent tasks. MCP for connecting Claude to external services in a repeatable, structured way.

Visual: Decision flowchart: three questions (building an app? local task? connecting to external service?) pointing to API/CLI/MCP respectively

Target words: ~130w

Beat -- CTA Bridge

Point: You now understand how AI connects to tools. Next: AI Agents Explained -- how those connections combine into agents that actually do things autonomously.

Visual: Preview clip from AI Agents Explained

Target words: ~60w

ANALOGY BANK

- API: Like a waiter at a restaurant -- you give your order (the request) in a specific format, the kitchen makes it (processes it), and the waiter brings back the result. Standardised interface, structured exchange.

- MCP vs API: Like a universal remote vs a custom remote for one TV -- the API is built specifically for that service. MCP is a universal standard that works across different services the same way, so Claude learns one interface that applies everywhere.

THESIS LINE LOCATION

Beat 5 -- The insight is: API, CLI, and MCP are not competing standards -- they solve different problems, and knowing which to reach for in which context is what separates someone who is scripting from someone who is building systems.

FRAMEWORK PHRASE

Right access, right job

SPECIFICITY BANK

- Anthropic API endpoint: api.anthropic.com
- MCP introduced by Anthropic in late 2024 as an open standard
- CLI benchmark: 100% reliable at 1,365-8,750 tokens per task (Scalekit 2025 benchmark)
- MCP benchmark: 72% reliable at 32,000-82,000 tokens per task -- higher cost and lower reliability for simple tasks
- Claude Code can call CLI tools natively from a session
- MCP servers can be added to Claude Desktop and Claude Code via config

CONSTRAINTS

- Do not go into authentication, OAuth, or API key management in depth
- Do not build an MCP server in this video -- keep it conceptual
- Do not explain n8n or other workflow platforms -- keep it Claude-native
- This is a concepts video -- no live coding, just demos and diagrams

CTA DIRECTION

You know how AI connects to tools. Next: AI Agents Explained -- how those tool connections combine into agents that actually do things without you directing every step.

PERSONAL ANGLE [DANIEL TO FILL]

Suggested: The first time you connected Claude to an external tool via API or MCP and it worked -- what the task was and what that capability unlocked for you.

VULNERABILITY MOMENT [DANIEL TO FILL]

Suggested: Over-engineering a simple task with MCP when a plain CLI call would have been faster, cheaper, and more reliable -- the lesson about right tool for the job.

Note: belongs in Beat 4